

TTB Label Verifier

Approach and design

Matthew Nolan · June 2026 · [live demo](#) · github.com/Mjnolan91/ttb-label-verifier

1. Summary

A compliance agent at TTB, the Alcohol and Tobacco Tax and Trade Bureau, uploads photos of an alcohol label. A vision model reads them into the full set of TTB-required fields, and deterministic code checks what is printed against what the application claims and against TTB's labeling rules: the brand name, the alcohol content within the legal tolerance for its beverage class, the statutory government warning word for word and correctly formatted, and the rest of the application field by field. The verdict is Approve, Needs review, or Reject, with a card per field explaining how it was compared. A batch screen runs the same engine over a few hundred labels at once and streams results into a review worklist; every read exports as JSON or CSV.

One asymmetry runs through the whole design: an unnecessary review costs an agent a few minutes, while a false approval costs trust in the tool. So uncertainty always routes to a person, and the build fails if approve precision drops below 98 percent on the labeled evaluation cases. The numbers I held myself to: 811 tests run offline in seconds with no API keys, the evaluation harness scores its 27 labeled cases (clean labels and deliberate defects both) at 100 percent, and the deployed demo measured a median of 3.1 seconds end to end, p95 5.2 paid by the serverless cold start, against the brief's roughly five-second ceiling. The live demo runs a real vision model, so it reads any label photo, and the verify screen loads a bundled sample pair with one click; that front-and-back sample reads in six to nine seconds live, the price of the dense two-image path.

2. What the customers said they wanted

The discovery notes are four voices, and each one set a requirement I treated as hard.

Sarah, the Deputy Director, said results must come back in about five seconds: the previous scanning vendor took 30 to 40 seconds per label and was abandoned. She also described who would use it (her agents range from fresh graduates to a 73-year-old she considers the benchmark user) and the workload spikes: importers dump 200 to 300 applications at once.

Marcus, in IT, explained that the outbound firewall had blocked the last vendor's ML endpoints, that they are an Azure shop, and that this prototype should stand alone, with no PII stored and no integration with COLA (TTB's label-approval filing system).

Dave, a 28-year agent, gave the false-rejection warning: STONE'S THROW and Stone's Throw are obviously the same product, and a tool doing rigid pattern matching would flood his queue with false rejections.

Jenny, a junior agent, described the strictest check: the government warning must match the statute word for word, with GOVERNMENT WARNING: in capital letters and bold; title case gets rejected. And badly photographed labels are out of scope but should fail gracefully.

So the requirement list: a hard five-second latency budget, a screen a 73-year-old can use without hunting, batch at importer scale, extraction that survives the firewall by running in-

tenant, tolerant brand matching with a review state, a strict statutory warning check, an end to retyping label fields by hand, and graceful failure on bad photos. Jenny's last line shaped the design most. I read "fail gracefully" as the error model for the whole tool: it is allowed to say it is not sure; it is never allowed to be silently wrong.

3. How I implemented it

One rule organizes the codebase: AI extracts, code decides. The vision model has exactly one job, transcribing the label images into structured fields with a confidence on each. Every pass, review, or fail decision after that is plain, unit-tested TypeScript: pure comparators for the field-by-field match (tolerant of case and punctuation, so `STONE'S THROW` and `Stone's Throw` compare equal, and a near miss routes to review instead of reject), a completeness matrix encoding what each beverage class must carry, and the statutory constants (the 27 CFR 16.21 warning text, the per-class alcohol tolerance bands) in one hand-verified module guarded by tests. When the tool rejects a label, the reason is a rule with a CFR citation, not a model's opinion.

A product's front and back ride one request as separate full-resolution image parts, so the model reads the panels together (brand on the front, warning on the back). I rejected the obvious alternative, stitching the photos into one composite, on resolution math: vision APIs cap total image resolution, so stitching halves each label's pixels exactly where the fine print lives. Each product is read several times in parallel, and a field's confidence is the fraction of samples that agree, a number I trust far more than a model's self-reported confidence. Cosmetic differences cluster as one reading; numeric differences never do. Fields that stay uncertain get bounded escalations, each allowed to clear a false alarm and never to flip a conflict to pass. A rescue re-reads only the doubtful fields on a second, stronger model. A dedicated judge on that same strong model decides whether the warning prefix is printed in bold; the answer is tri-state, so "could not verify" routes to review rather than silently passing. And a warning still missing after that gets a focus pass that finds it in any orientation, crops, derotates, and upscales the region, and judges again from the zoomed crop.

The single screen leads with the application comparison, per the brief. The AI's reading pre-fills every application input as a gray suggestion the agent accepts with Tab or one click, which removes the re-keying the agents complained about. One screen, nothing to hunt for; that was the 73-year-old test. A suggestion only counts once a person accepts it, so the model never approves its own work. The batch screen runs the same engine over a folder of images: fronts and backs pair by filename, an optional CSV supplies the claimed values, rows stream in and stay reviewable (automatic retry on transient failures, application values editable in place, and a delayed second look that re-reads exactly the fields a clean read missed), and a header toggle switches between the two modes. Both screens and both exports derive from one shared field catalog, so they cannot drift apart.

The whole suite is offline and deterministic. The default provider is a mock keyed off fixture filenames, so the 811 tests in 63 files and the 27-case evaluation run in seconds with zero keys, and CI runs the same gate (typecheck, lint, test, eval, build) on every push to main. The eval is a gate, not a report: clean labels plus deliberately broken ones (a title-case warning, an out-of-tolerance ABV, a brand typo, a missing warning, an unreadable photo that must never auto-approve), with the build failing under a 98 percent approve-precision floor.

4. Why I chose this architecture

The AI models. The deployed demo runs OpenAI gpt-4.1 for extraction, with gpt-5.5 reserved for the narrow judgments (the warning's bold prefix, the focused re-reads of doubtful fields), and I arrived at that split by measurement. Moving extraction itself to the strong model more than doubled the median latency (2.8 to 6.5 seconds in that A/B) for identical verdicts; the same experiment on Gemini also tripped the strong model's 25-requests-per-minute quota, failing 4 of 9 requests outright. So the fast model transcribes, and the strongest model is spent on the one judgment that can hard-fail a label. That rule held on both vendors I measured, and the in-tenant Azure path serves the same model family behind the same interface.

One provider interface, six modes. Extraction sits behind a VisionProvider interface with mock, OpenAI, Gemini, Azure OpenAI, Azure Document Intelligence, and an ensemble mode that routes cross-provider disagreement to review. That shape is Marcus's firewall requirement made concrete: the production path is the in-tenant Azure pair, the public demo runs on a single OpenAI key, and switching providers is an env-var change. The offline mock is what keeps the test suite and CI hermetic.

Why Next.js. One TypeScript codebase carries the UI and the API routes, so the model keys stay server-side and there is no second service to build, deploy, or keep in sync. The route handlers run the extraction pipeline, server components render the screens, and the standalone output ships as a small container for the Azure target. Strict TypeScript end to end also means the extracted-field shape is one set of types shared by the pipeline, the UI, and the CSV export. I wanted the plumbing boring and reviewable so the interesting code could be the domain logic.

Why Vercel for the demo. I wanted a reviewer inside the running app one click after opening the repo, and Vercel auto-deploys this repo's main branch with zero configuration. It is deliberately just the demo host: the firewall requirement points production at in-tenant Azure, which is why the repo ships a multi-stage Dockerfile with documented App Service and Container Apps paths. Vercel is the shortest path from the repo to a working demo in your browser; Azure is where it would actually live.

Why deterministic comparators instead of letting the model judge. A compliance verdict has to be auditable and reproducible. "The model felt the brand matched" is not an answer a government reviewer can act on, so the model never makes the final call. The statutory warning text lives in one constant guarded by a verbatim unit test, and the check is a strict character-for-character comparison against it.

What I would do next. Deploy the container in-tenant with provisioned model quota behind the agency gateway (authentication and rate limiting), then layout-preserving OCR to unlock the type-size and placement rules that text-only extraction cannot check, then a sampled human-audit loop over auto-approvals so the false-approval rate keeps getting measured once real labels flow through it. COLA integration goes last; reaching that system is procurement work before it is engineering work.